

Machine Learning

Introduction to ML Model Evaluation

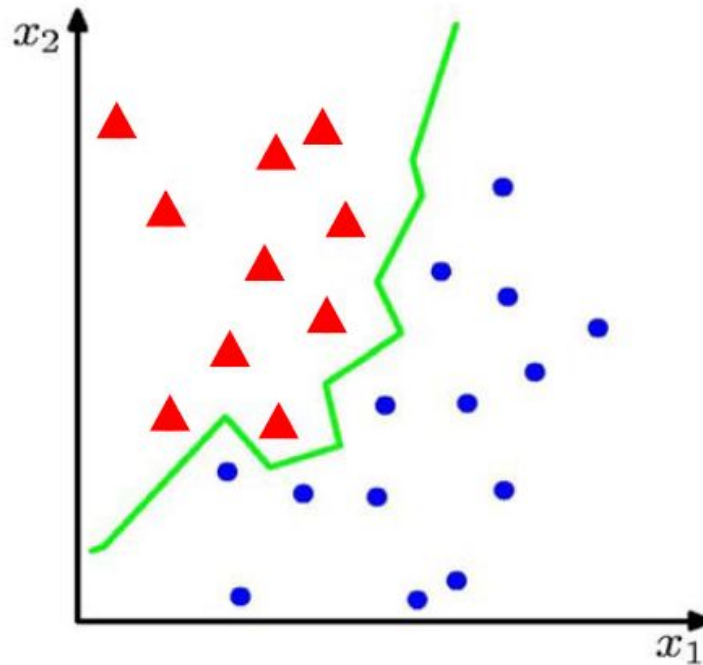
IIIT Sri City



This week's Agenda

- Recap to classification?
- How to evaluate the classification model?
- Recap to regression problem?
- How to evaluate the regression model?
- Evaluation Metrics for a regression model
- Dataset: Train, Validation and Test sets
- Train, test and validation split
- Overfit and Underfit

Recap: What is classification problem?



- Suppose we are given a training set of N observations

$(x_1, \dots, x_N)$ and $(y_1, \dots, y_N)$, $x_i \in \mathbb{R}^d$, $y_i \in \{-1, 1\}$

- Classification problem is to estimate $f(x)$ from this data such that

$$f(x_i) = y_i$$

Evaluating a classifier

Classification Accuracy and Classification Error

Let the test set size be N .

Classification Accuracy (CA):

Number of correct predictions

$$CA = \frac{\text{Number of correct predictions}}{N}$$

Number of wrong predictions

$$\text{Error} = \frac{\text{Number of wrong predictions}}{N}$$

CA and Error

In the previous formula we assumed that there is a finite test set to evaluate the classifier.

But this is not the actual accuracy or error of the classifier. **Actual accuracy or error has to consider all data elements from the feature space.** Practically this may not be possible. But, in theory one can find this.

Recap: Probability

Probability theory is built on 3 axioms.

The terms one should be clear:

1. Experiment
2. Sample space
3. Event
4. Probability

Kolmogorov Axioms of Probability

To the probabilities of outcomes/events of an experiment must obey the axioms:

- **Axiom 1:** For any event A , $\Pr(A) \geq 0$
- **Axiom 2:** $\Pr(\Omega) = 1$
- **Axiom 3:** For a collection of mutually exclusive events, $A_1, A_2, \dots, A_n$

$$\Pr(A_1 \cup A_2 \dots \cup A_n) = \sum_{i=1}^n \Pr(A_i)$$

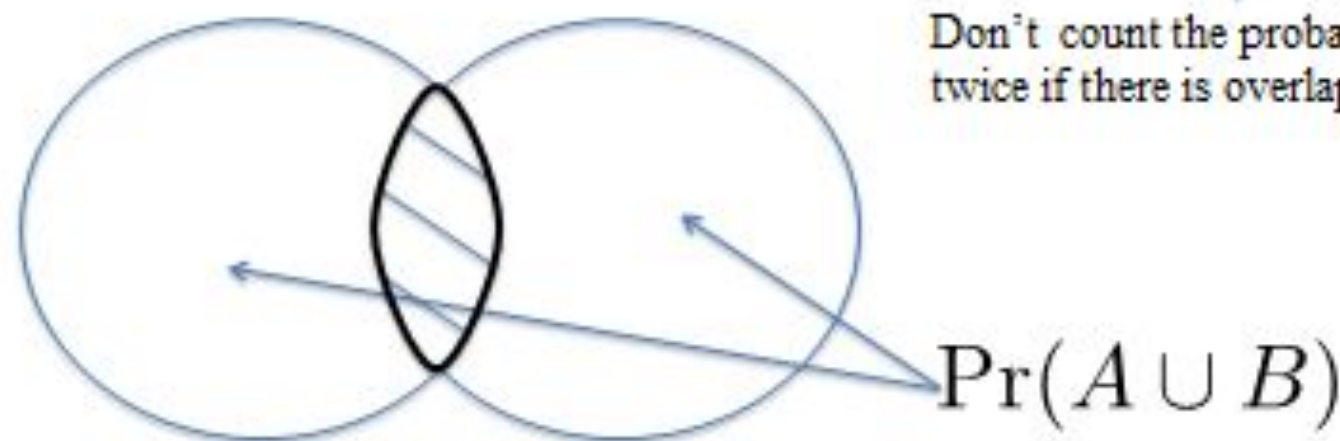
- Everything else in probability theory can be deduced starting with these axioms

Handy Consequences of Kolmogorov Axioms

- Important consequences:
 - Probability of a union of non-disjoint events

$$\Pr(A \cup B) = \Pr(A) + \Pr(B) - \Pr(A \cap B)$$

In words: The probability of A *or* B is the probability of A *plus* the probability of B *minus* the probability of A *and* B



Loss function

Let X be the given example with y being the target label (ground-truth).

Let the classifier predicted y'

Loss of this prediction, $P(\text{error} | X)$ and $P(\text{error})$:

$$L(x) = \begin{cases} 0, & \text{if } y = y' \\ 1, & \text{otherwise} \end{cases}$$

$$P(\text{error}|x) = L(x)$$

{This assumes that the feat. space
is a discrete one}

$$\begin{aligned} P(\text{error}) &= E[P(\text{error}|x)] \\ &= \sum_{x \in X} L(x) P(x) \end{aligned}$$

If the space is continuous,

$$p(\text{error}|x) = L(x)$$

$$P(\text{error}) = E[p(\text{error}|x)]$$

$$= \int L(x) p(x) dx$$

{The integration is over the feat-space}

Example (discrete case)

Data point	Ground truth	Predicted label	Probability
X1	Cat	Cat	1/8
X2	Cat	Dog	1/4
X3	Dog	Dog	1/4
X4	Dog	Cat	3/8

$$P(\text{error}) = \text{Average Error rate} = 0(1/8) + 1(1/4) + 0(1/4) + 1(3/8) = 5/8.$$

Note: The entire space is covered by these four examples.

Eg 2

Assume we are working with 1D, two class problem. Let the class labels are 1 and 2.

Let the classifier is a simple rule: if the feature value (x) is less-than 10 the class assigned is 1, else the class assigned is 2.

$$x < 10 \Rightarrow \text{class} = 1$$

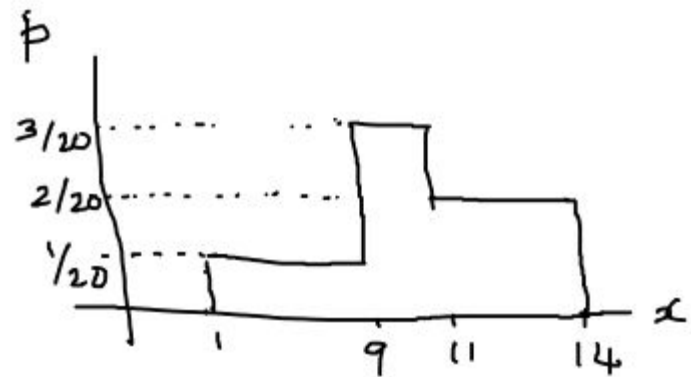
$$x \geq 10 \Rightarrow \text{class} = 2$$

Eg 2 Continued

Assume that the data is drawn from the mixture of the two class-conditionals (shown below) where each class is equally likely.



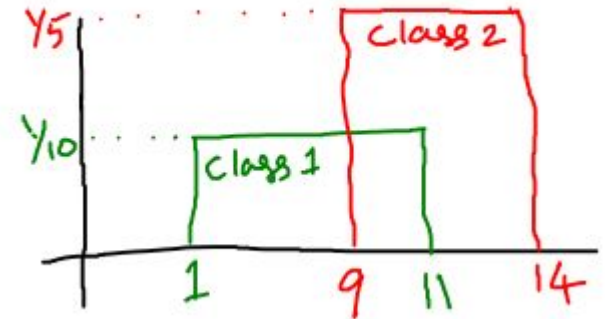
Class-conditional distributions



Distribution from which x is drawn

Eg 2

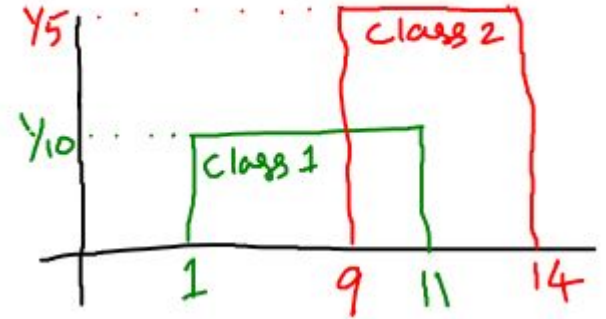
The classifier is not making any mistake for $x < 9$ and for $x > 11$. Only when x is in $[9, 11]$ one has to worry about the mistakes made.



Eg 2

[9, 10] error: Let X is an arbitrary element drawn in [9,10]

$$P(\text{err} \mid X) = P(y=\text{class2} \mid X)$$



We use the notation $P(\text{class2} \mid X)$ instead of $P(y=\text{class2} \mid X)$

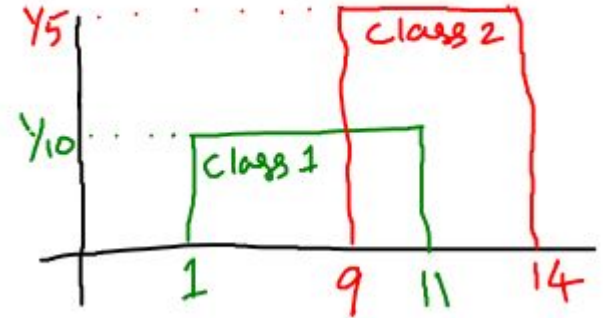
Eg 2

[9, 10] error: Let X is an arbitrary element drawn in [9,10]

$$P(\text{err} \mid X) = P(\text{class2} \mid X)$$

$$P(\text{class2} \mid X) = \frac{p(X, \text{class2})}{p(X)}$$

$$= \frac{p(X \mid \text{class2}) P(\text{class2})}{p(X)}$$



$$p(X) = p(X|\text{class1})P(\text{class1}) + p(X|\text{class2})P(\text{class2})$$

So, for X in $[9,10]$,

$$P(\text{class2} | X) = \frac{\frac{1}{5} \cdot \frac{1}{2}}{\frac{1}{5} \cdot \frac{1}{2} + (1/10) \cdot \frac{1}{2}} = \frac{2}{3}$$

[10,11] error: Let X is an arbitrary element drawn in [10,11]

$$\begin{aligned} P(\text{err} \mid X) &= P(\text{class1} \mid X) \\ &= 1/3 \end{aligned}$$

P(error)

$$P(\text{error}) = \int P(\text{error} | x) p(x) dx$$

$$= \int_1^9 P(\text{class 2} | x) p(x) dx$$

$$+ \int_9^{10} P(\text{class 2} | x) p(x) dx$$

$$+ \int_{10}^{11} P(\text{class 1} | x) p(x) dx$$

$$+ \int_{11}^{14} P(\text{class 1} | x) p(x) dx$$

$$P(\text{error}) = 0 + \frac{2}{3} \cdot (3/20) + \frac{1}{3} \cdot (3/20) + 0 = 3/20$$

Eg2 Contd

By shifting the threshold of the classifier can we reduce the error rate of the classifier?

If so, what would be the best possible threshold?

This actually points to the Bayes Classifier.

Actual error-rate

The actual error-rate calculation is theoretical and is possible when the distributions (Probability structure) of the given problem is known.

In reality we will not be having this.

Test set

Test set is an independently drawn set from the same distribution as the training set.

Since we are given with an example set, normal practice is to divide this into **the training set** and **the test set**.

Note that this division has to be random and in practice often disjoint division is followed.

Multiple test sets are possible

Since multiple training and test set divisions are possible, often we measure the accuracy or error as the average one over multiple divisions.

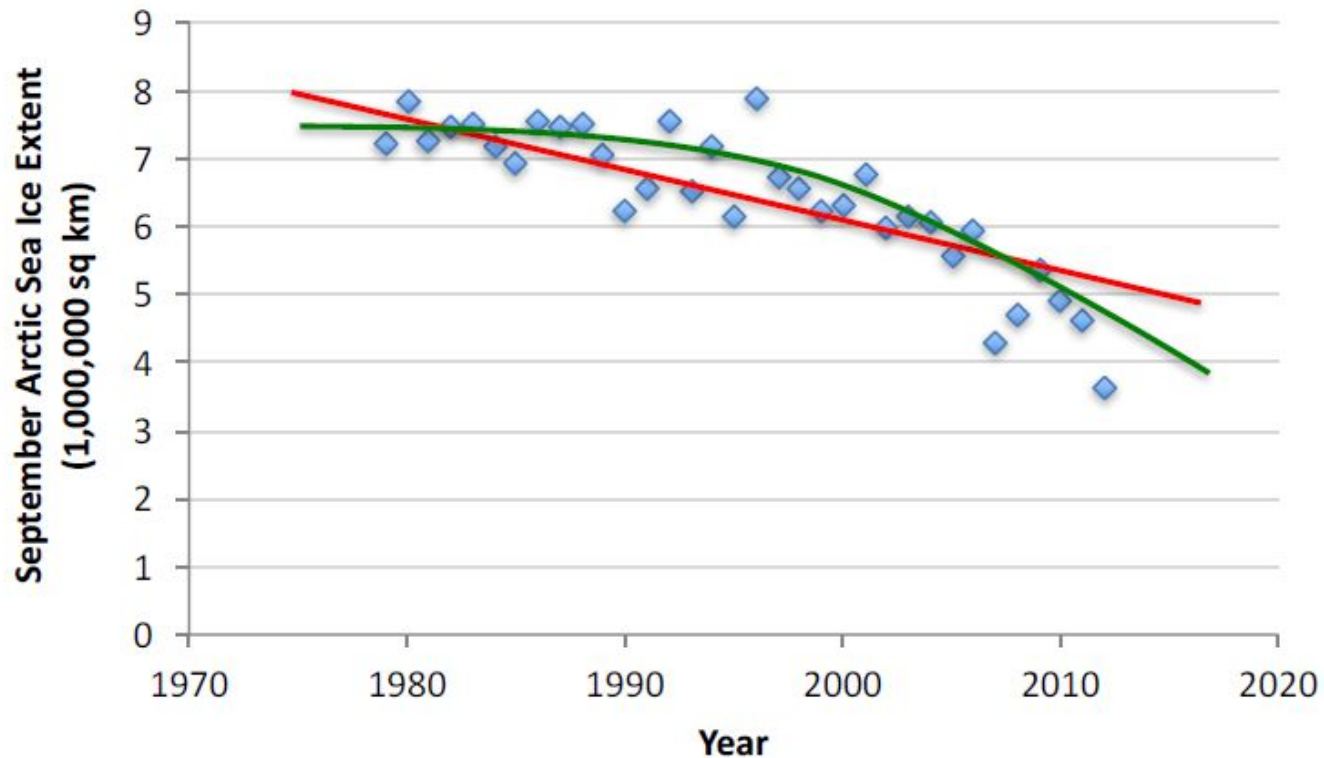
This can give us variance or standard deviation of the computed accuracy or error.

Later we will see about **bias and variance** of the learning.

Regression

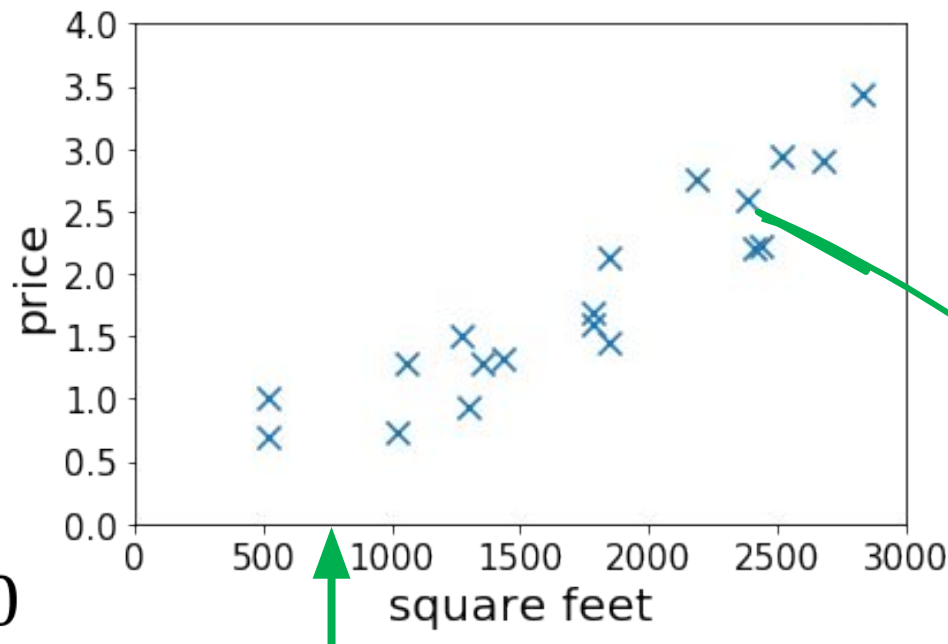
What is the Regression Problem?

- Given $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$
- Learn a function $f(x)$ to predict y given x
 - y is real-valued == regression



Housing Price Prediction

- Given: a dataset that contains n samples
 $(x^{(1)}, y^{(1)}), \dots (x^{(n)}, y^{(n)})$
- Task:** If a residence has x square feet, predict its price?

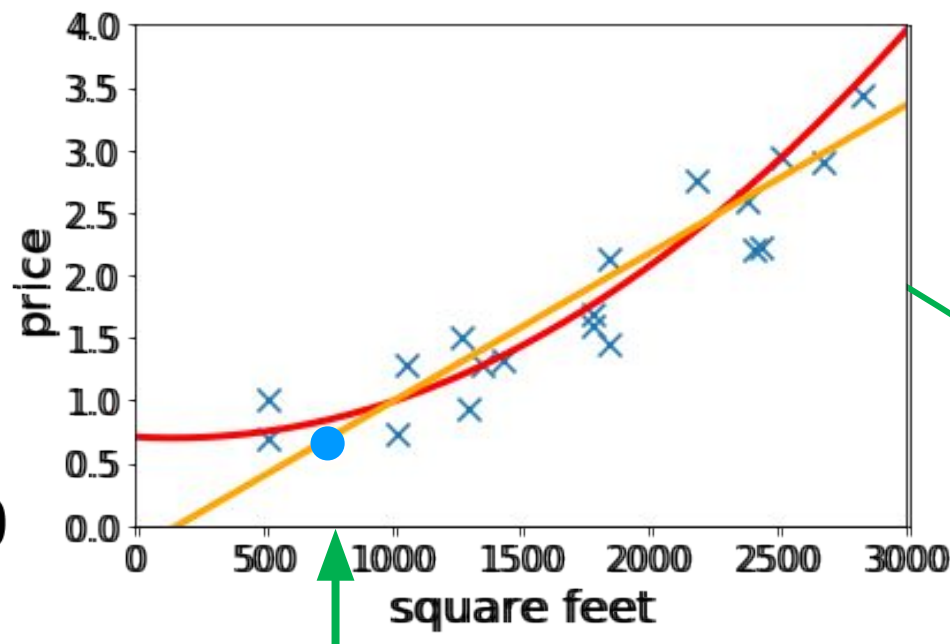


15th sample
 $(x^{(15)}, y^{(15)})$

$x = 800$
 $y = ?$

Housing Price Prediction

- Given: a dataset that contains n samples
 $(x^{(1)}, y^{(1)}), \dots (x^{(n)}, y^{(n)})$
- Task:** If a residence has x square feet, predict its price?



$x = 800$
 $y = ?$

15th sample
 $(x^{(15)}, y^{(15)})$

- Solution:** fitting linear/quadratic functions to the dataset.

How to evaluate the regression model?

- What is the accuracy of your model prediction?
- Compared to classification (where you have discrete set of labels as prediction) predicting the accuracy in regression (where you have continuous real value number) is slightly difficult.!
- It might be impossible for your ML model to predict the exact value.
- So to calculate the accuracy, you can compared the predicted value as how close it is against the real value.

Evaluation Metrics for a Regression model

- *There are three main metrics used for evaluating the regression models:*
 - *R Square/Square of the Correlation Coefficient*
 - *Mean Square Error(MSE)/Root Mean Square Error(RMSE)*
 - *Mean Absolute Error(MAE)*

Evaluating a regression model: R square value

- *R Square/coefficient of determination*: It measures the proportion of the variance in the dependent variable that is predictable from the independent variable(s).
- *R Square* value is between 0 to 1 and bigger value indicates a better fit between prediction and actual value

$$R^2 = 1 - \frac{SS_{Regression}}{SS_{Total}} = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

Where y_i is the original value, $\hat{y}_i$ is the predicted value and $\bar{y}$ is the mean of original values.

Evaluating a regression model: Mean Squared Error (MSE)

- MSE is calculated by the sum of square of prediction error which is real output minus predicted output and then divide by the number of data points.
- It gives how much your predicted results deviate from the actual value (in terms of square of the deviation).
- Root Mean Square Error(RMSE) is the square root of MSE.

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

Where y_i is the original value and $\hat{y}_i$ is the predicted value.

Evaluating a regression model: Mean Absolute Error (MAE)

- Mean Absolute Error(MAE) is similar to Mean Square Error(MSE).
- Unlike MSE where we take the sum of square of errors, MAE computes the sum of absolute value of error.

$$MAE = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

Where y_i is the original value and $\hat{y}_i$ is the predicted value.

- Note: MSE gives larger penalisation to big prediction error by squaring it while MAE treats all errors the same.

Dataset: Train, Validation and Test sets

- **Train set:** The model is initially fit on a training dataset, which is a set of examples used to fit the parameters (e.g. weights of connections between neurons in artificial neural networks) of the model.
- In practice, the training dataset often consists of pairs of an input vector (or scalar) and the corresponding output vector (or scalar), where the answer key is commonly denoted as the target (or label).

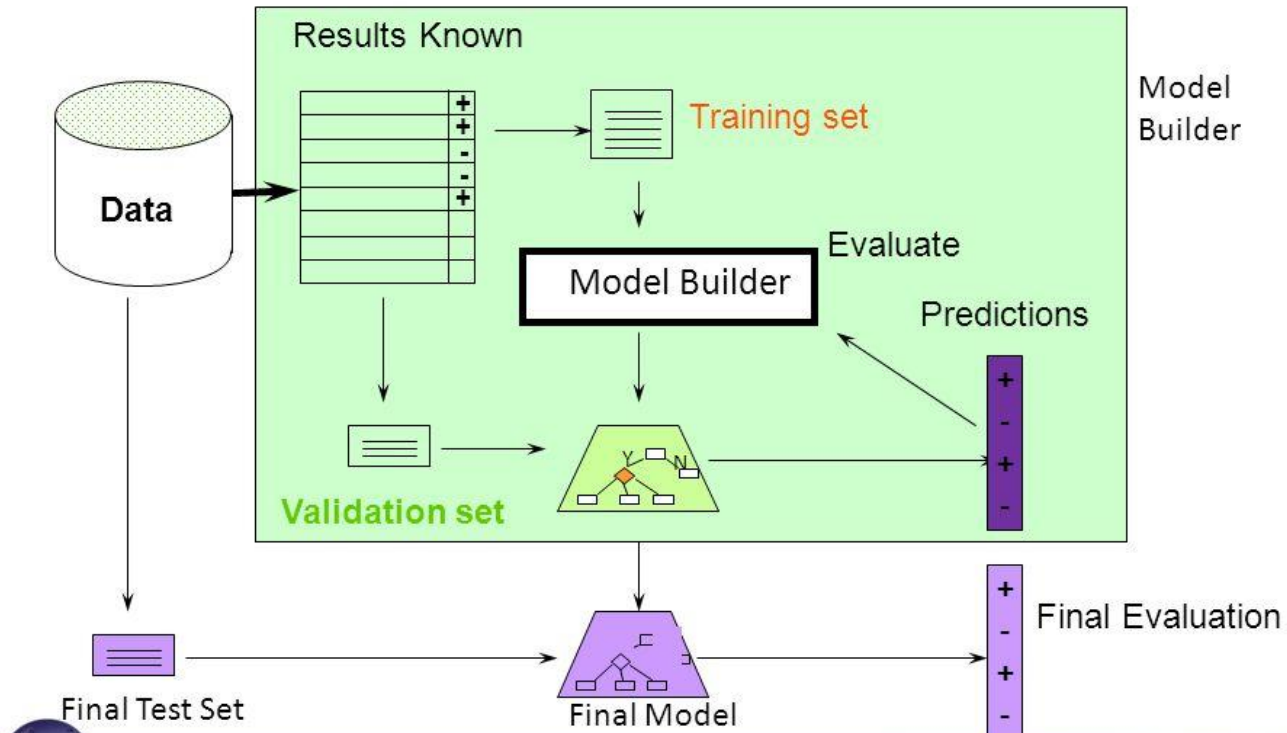
Dataset: Train, Validation and Test sets

- **Validation set:** Successively, the fitted model is used to predict the responses for the observations in a second dataset called the validation dataset.
- The validation dataset provides an unbiased evaluation of a model fit on the training dataset while tuning the model's hyperparameters e.g. the number of hidden units (layers and layer widths) in a neural network.
- The validation dataset functions as a hybrid: it is training data used for testing, but neither as part of the low-level training nor as part of the final testing.

Dataset: Train, Validation and Test sets

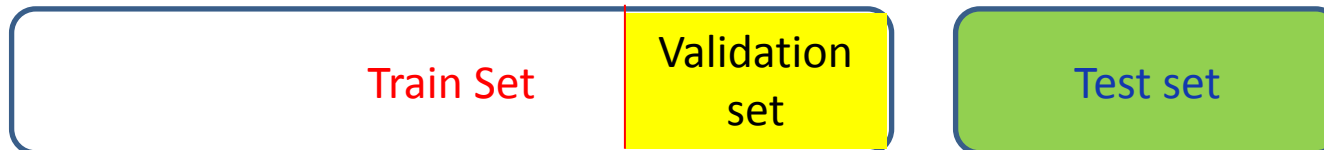
- **Test set:** Finally, the test dataset is a dataset used to provide an unbiased evaluation of a final model fit on the training dataset.
- The test dataset is typically used to assess the final model that is selected during the validation process.
- It is only used once a model is completely trained(using the train and validation sets).
 - Many a times the validation set is used as the test set, but it is not good practice.
- The test set should be drawn independently from the same distribution as the training set.

Train, test and validation split



Data Sampling Methods

- **Hold out method:** In the holdout method, we randomly assign data points to two sets d_0 and d_1 , usually called the training set and the test set, respectively.
- The size of each of the sets is arbitrary although typically the test set is smaller than the training set.
- A smaller part of the training set forms the validation set. As the training progresses this validation set may vary (may be other part of the training set)



Data Sampling Methods

- **Cross Validation Method:** Cross-validation involves partitioning a sample of data into complementary subsets, performing the analysis on one subset (called the training set), and validating the analysis on the other subset (Validation set).
- To reduce variability, in most methods multiple rounds of cross-validation are performed using different partitions, and the validation results are combined (e.g. averaged) over the rounds to give an estimate of the model's predictive performance.

$n = 8$



Validation



Train

Model 1



Data Sampling Methods

k-fold Cross validation: This procedure has a single parameter called “k” that refers to the number of groups that a given data sample is to be split into.

The general procedure is as follows:

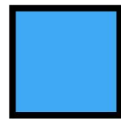
1. Shuffle the dataset randomly.
2. Split the dataset into k groups
3. For each unique group:
 1. Take the group as a hold out or validate data set
 2. Take the remaining groups as a training data set
 3. Fit a model on the training set and evaluate it on the validation set
 4. Retain the evaluation score and discard the model
4. Summarize the skill of the model using the sample of model evaluation scores

k-fold Cross validation:

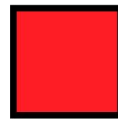
$n = 12$

$k = 3$

Data



Validation



Train



Overfit vs Underfit

- **Overfitting:** refers to a model that models the training data too well.
- Overfitting happens when a model learns the detail and noise in the training data to the extent that it negatively impacts the performance of the model on new data.
- In overfitting, ML model learns the inference rules based on the noise or random fluctuations in the training data.

Overfit vs Underfit

- **Underfitting:** refers to a model that can neither model the training data nor generalize to new data.
- An underfit machine learning model is not a suitable model and will be obvious as it will have poor performance on the training data.
- Underfitting is often not discussed as it is easy to detect given a good performance metric. The remedy is to move on and try alternate machine learning algorithms.

**Thank You:
Question?**